

EJERCICIO DE REPASO DE USO DE CLASES GENERICAS

PLANTILLA DE CLASES (TEMA 2)

El siguiente main() hace uso del TAD pila a través de las clases **Pila** y **PilaDescriptiva**:

```
#include <iostream>
#include <cstdlib>
#include "Pila.h"
#include "PilaDescriptiva.h"

using namespace std;

int main() {
    Pila p; //crea una pila dinámica de enteros con una capacidad inicial de 4 elementos
            //se puede indicar un tamaño diferente Ej: Pila p(8); crea una pila de tamaño 8
    Pila q(6); //crea una pila dinámica de enteros de capacidad inicial para 6 elementos
    cout << "La pila tiene " << p.longitud() << " elementos" << endl; //indica cuantos elementos hay
    cout << "y una capacidad de " << p.capacidad() << " elementos\n"; //indica cuantos caben
    cout << "p: ";
    p.ver(); //muestra los elementos de la pila entre corchetes
    cout << endl;
    cout << "p: " << p << endl; //muestra los elementos de la pila
    for(int i=1;i<6;i++) {
        p.apilar(i); //mete un elemento en la pila
    }
    cout << "p: "; p.ver(); cout << endl;
    cout << "p: " << p << endl;
    cout << p.cima() << endl; //devuelve el elemento de la cima
                                //de la pila (si no hay ERROR)
    Pila r(p); //crea una pila idéntica a la pila p

    Pila d; d.apilar(2); d.apilar(5);
    cout << "Pilas r y d: "; Pila::mostrarPilas(r,d); cout << endl;
    cout << "Cima de r: "; Pila::verCimaDe(r); cout << endl;
    cout << "Cima de d: "; Pila::verCimaDe(d); cout << endl;

    p.desapilar(); //elimina el ultimo elemento metido en la pila
    p.desapilar();

    q=p; //hace que la pila q contenga lo mismo que la pila p
    q.apilar(9);
    r.apilar(8);

    cout << "p: "; p.ver(); cout << endl;
    cout << "p: " << p << endl;
    cout << p.cima() << endl;
    while (!p.esvacía()) { //indica si la pila esta vacia o no
        p.desapilar();
    }
    cout << "p: "; p.ver(); cout << endl;
    cout << "q: "; q.ver(); cout << endl;
    cout << "r: "; r.ver(); cout << endl;

    PilaDescriptiva pd("Puntos"); //crea una pila dinámica con una capacidad inicial de 4 int
                                //el texto describe el contenido de la pila
    PilaDescriptiva qd(7); //crea una pila dinámica de enteros de capacidad inicial para 7 elementos
                            //el texto explicativo es opcional (si no se indica será "Pila")
    cout << "La pila pd y qd tienen " << pd.longitud() << " y " << qd.longitud() << " items" << endl;
    cout << "y una capacidad de " << pd.capacidad() << " y " << qd.capacidad() << " elementos\n";
    cout << "pd es "; pd.ver(); cout << endl;
    cout << "qd es " << qd << endl;
    if (!pd.esvacía())
        cout << pd.cima();
    for(int i=1;i<6;i++) {
        pd.apilar(i);
    }
    PilaDescriptiva rd(pd); //crea una pila idéntica a la pila pd

    pd.desapilar();pd.desapilar();
    qd=pd;
    qd.desapilar();
    cout << pd << endl; //muestra la descripcion de la pila: y muestra su contenido
    cout << "pd es "; pd.ver(); cout << endl; //muestra descripcion y el contenido entre corchetes
    cout << "qd es "; qd.ver(); cout << endl;
```

SALIDA POR PANTALLA

```
La pila tiene 0 elementos
y una capacidad de 4 elementos
p: []
p: (vacío)
p: [1 2 3 4 5]
p: 1 2 3 4 5
5
Pilas r y d: [1 2 3 4 5],[2 5]
Cima de r: 5
Cima de d: 5
p: [1 2 3]
p: 1 2 3
3
p: []
q: [1 2 3 9]
r: [1 2 3 4 5 8]
La pila pd y qd tienen 0 y 0 items
y una capacidad de 4 y 7 elementos
pd es Puntos:[]
qd es Pila:(vacío)
Puntos: 1 2 3
pd es Puntos:[1 2 3]
qd es Puntos:[1 2]
rd es Puntos:[1 2 3 4 5]
Puntos:[1 2],Puntos:[1 2 3 4 5]
3
```

```

cout << "rd es "; rd.ver(); cout << endl;

PilaDescriptiva::mostrarPilas(qd, rd); cout << endl;
cout << pd.cima() << endl;
return 0;
}

```

Implementa la clase **Pila**, que simula el funcionamiento de una pila dinámica de capacidad ilimitada, y sus métodos asociados de forma que se pueda ejecutar el **main()** anterior produciendo la salida indicada por pantalla (la descripción de lo que hace cada método viene comentada en el código y la salida que genera cada método es la que aparece en el recuadro emergente). Aplicando herencia implementa también la clase **PilaDescriptiva** (una pila descriptiva es una pila que permite tener asociado una cadena de caracteres que describe el contenido de dicha pila).

Tanto la clase **Pila** como la clase **PilaDescriptiva** permiten copiar el contenido de una pila en otra pila utilizando el operador de asignación y permiten crear una pila a partir de otra.

Realiza el ejercicio considerando que:

- Las clases **Pila** y **PilaDescriptiva** simulan el comportamiento de una pila dinámica de **enteros** de capacidad ilimitada.
- Las clases **Pila** y **PilaDescriptiva** simulan el comportamiento de una pila dinámica **genérica** (válida para **cualquier tipo de datos**) de capacidad ilimitada. Usa plantilla de clases y especializaciones de plantillas de clases y/o de métodos para hacer que la clase genérica **Pila** y **PilaDescriptiva** sirva para crear pilas de **int**, **float** y **Complejo** teniendo en cuenta que la clase **Complejo** no se puede modificar y contiene los métodos indicados a continuación.

```

using namespace std;

class Complejo {
    int real;
    int imaginario;
public:
    Complejo(int real=0, int imaginario=0);
    virtual ~Complejo() { }
    int getr() const { return real; }
    void set(int r, int i) { real=r; imaginario=i; }
    int geti() const { return imaginario; }
    void ver() const;
};

Complejo::Complejo(int real, int imaginario) {
    this->real=real;
    this->imaginario=imaginario;
}

void Complejo::ver() const {
    if (imaginario >= 0)
        cout << real << "+" << imaginario << "i";
    else
        cout << real << imaginario << "i";
}

```

- Indica que método o función habría que añadir a la clase **Complejo** para que en la clase genérica **Pila** y **PilaDescriptiva** creada en el apartado anterior **no fuera necesario especializar nada**.

SOLUCION:

- a) Las clases **Pila** y **PilaDescriptiva** simulan el comportamiento de una pila dinámica de **enteros** de capacidad ilimitada.

Para crear una pila dinámica de capacidad ilimitada debemos usar arrays dinámicos

```
#include <iostream>
#include <cstdlib> //system("pause");
#include <cstring> //strlen, strcmp, strcpy

using namespace std;

class Pila {
    int *t; //tabla dinamica de elementos
    int n;   //numero de elementos del array
    int nmax; //capacidad del array
protected:
    int ampliar(); //método para duplicar el tamaño del array dinamico apuntado por t
public:
    Pila(int tam=4); //tamaño opcional, si no se indica ningun valor se toma tamaño 4
    virtual ~Pila(); //destructor
    Pila(const Pila& p); //constructor de copia
    Pila& operator=(const Pila& p); //operador de asignacion

    int longitud() const { return n; }
    int capacidad() const { return nmax; }

    void apilar(int valor);
    void desapilar();
    int cima() const;
    bool esvacia() const { return n==0; }

    virtual void ver() const;

    static void mostrarPilas(const Pila &p1, const Pila &p2); //metodo estatico
    static void verCimaDe(const Pila &p); //metodo estatico

    friend ostream& operator<<(ostream& s, const Pila &p);
};

ostream& operator<<(ostream& s, const Pila &p);

Pila::Pila(int tam) {
    t=new int[tam];
    nmax=tam;
    n=0;
}

Pila::~Pila() { delete [] t; }

Pila::Pila(const Pila& p) {
    t=new int[p.nmax];
    nmax=p.nmax;
    n=p.n;
    for (int i=0; i<n; i++) {
        t[i]=p.t[i];
    }
}

Pila& Pila::operator=(const Pila& p) {
    if (this != &p) {
        delete [] t;
        t=new int[p.nmax];
        nmax=p.nmax;
        n=p.n;
        for (int i=0; i<n; i++)
            t[i]=p.t[i];
    }
    return *this;
}
```

```

void Pila::apilar(int valor) {
    if (n==nmax)
        ampliar();
    t[n]=valor;
    n++;
}

void Pila::desapilar() {
    if (n==0) {
        cout << "Error: No hay elementos" << endl;
        system("pause");
        exit(1);
    }
    n--;
}

int Pila::cima() const {
    if (n==0) {
        cout << "Error: No hay elementos" << endl;
        system("pause");
        exit(1);
    }
    return t[n-1];
}

int Pila::ampliar() {
    int *aux=new int[nmax*2];
    for(int i=0; i<n; i++)
        aux[i]=t[i];
    delete [] t;
    t=aux;
    nmax*=2;
    return nmax;
}

void Pila::ver() const {
    cout << "[";
    if (n>0) {
        for(int i=0; i<n-1; i++)
            cout << t[i] << " ";
        cout << t[n-1];
    }
    cout << "]";
}

void Pila::mostrarPilas(const Pila &p1, const Pila &p2) { //método estatico
    p1.ver(); cout<< ",";
    p2.ver();
}

void Pila::verCimaDe(const Pila &p) { //método estatico
    cout << p.cima();
}

ostream& operator<<(ostream& s, const Pila &p) {
    if (p.n==0)
        s << "(vacio)";
    for(int i=0; i<p.n; i++) {
        s << p.t[i] << " ";
    }
    return s;
}

```

```

class PilaDescriptiva: public Pila {
    char *descripcion;
public:
    PilaDescriptiva(int tam, char *texto="Pila");
    PilaDescriptiva(char *texto="Pila");
    virtual ~PilaDescriptiva();
    PilaDescriptiva(const PilaDescriptiva& p);
    PilaDescriptiva& operator=(const PilaDescriptiva& p);

    virtual void ver() const;
    friend ostream& operator<<(ostream& s, const PilaDescriptiva &p);
};

ostream& operator<<(ostream& s, const PilaDescriptiva &p);

PilaDescriptiva::PilaDescriptiva(int tam, char *texto): Pila(tam) {
    descripcion=new char[strlen(texto)+1];
    strcpy(descripcion, texto);
}

PilaDescriptiva::PilaDescriptiva(char *texto): Pila() {
    descripcion=new char[strlen(texto)+1];
    strcpy(descripcion, texto);
}

PilaDescriptiva::~PilaDescriptiva() {
    delete [] descripcion;
}

PilaDescriptiva::PilaDescriptiva(const PilaDescriptiva& p): Pila(p) {
    descripcion=new char[strlen(p.descripcion)+1];
    strcpy(descripcion, p.descripcion);
}

PilaDescriptiva& PilaDescriptiva::operator=(const PilaDescriptiva& p) {
    if (this != &p) {
        Pila::operator=(p);
        delete [] descripcion;
        descripcion=new char[strlen(p.descripcion)+1];
        strcpy(descripcion, p.descripcion);
    }
    return *this;
}

void PilaDescriptiva::ver() const {
    cout << descripcion << ": ";
    Pila::ver();
}

ostream& operator<<(ostream& s, const PilaDescriptiva &p) {
    s << p.descripcion << ": " << (Pila &) p;
    return s;
}

```

- b) Las clases `Pila` y `PilaDescriptiva` simulan el comportamiento de una pila dinámica **genérica** (válida para **cualquier tipo de datos**) de capacidad ilimitada. Usa plantilla de clases y especializaciones de plantillas de clases y/o de métodos para hacer que la clase genérica `Pila` y `PilaDescriptiva` sirva para crear pilas de **int**, **float** y **Complejo** teniendo en cuenta que la clase **Complejo** no se puede modificar y contiene los métodos indicados a continuación.

```
using namespace std;

class Complejo {
    int real;
    int imaginario;
public:
    Complejo(int real=0, int imaginario=0);
    virtual ~Complejo() { }
    int getr() const { return real; }
    void set(int r, int i) { real=r; imaginario=i; }
    int geti() const { return imaginario; }
    void ver() const;
};

Complejo::Complejo(int real, int imaginario) {
    this->real=real;
    this->imaginario=imaginario;
}

void Complejo::ver() const {
    if (imaginario >= 0)
        cout << real << "+" << imaginario << "i";
    else
        cout << real << imaginario << "i";
}
```

Crearemos 2 clases genéricas `Pila<T>` y `PilaDescriptiva<T>`.

Todos los métodos implementados anteriormente pueden servir para los tipos **int**, **float** y **Complejo** excepto el método `Pila::ver()` y la función amiga `operator<<` ya que dichos métodos utilizan el operador `<<` para mostrar el contenido de los elementos de la pila.

```
void Pila::ver() const { //codigo anterior sin plantillas
    cout << "[";
    if (n>0) {
        for(int i=0; i<n-1; i++)
            cout << t[i] << " ";
        cout << t[n-1];
    }
    cout << "]";
}

ostream& operator<<(ostream& s, const Pila &p) { //codigo anterior sin plantillas
    if (p.n==0)
        s << "(vacio)";
    for(int i=0; i<p.n; i++) {
        s << p.t[i] << " ";
    }
    return s;
}
```

Como la clase **Complejo** no tiene implementado el operador `<<` debemos especializar dichos métodos para cuando el tipo de dato es **Complejo** en la clase `Pila<T>`.

Respecto a los métodos estáticos cuyo código hecho en el apartado anterior es el siguiente:

```
void Pila::mostrarPilas(const Pila &p1, const Pila &p2) { //codigo anterior sin plantillas
    p1.ver(); cout<< ", "; p2.ver();
}

void Pila::verCimaDe(const Pila &p) { //codigo anterior sin plantillas
    cout << p.cima();
}
```

Como la clase Complejo no implementa el operador << debemos especializar verCimaDe() para cuando el tipo de dato es Complejo en la clase Pila<T>.

Si consideramos que los parámetros de estos métodos estáticos pueden ser de distinto tipo entre sí y no tienen por qué coincidir con el tipo de la Pila genérica la plantilla de Pila genérica para estos métodos estáticos sería algo así:

```
template <class T>
class Pila {
...
    template <class H1, class H2> static void mostrarPilas(const Pila<H1> &p1, const Pila<H2> &p2);
    template <class H> static void verCimaDe(const Pila<H> &p);
...
};
```

El método verCimaDe() hay que especializarlos para cuando el tipo de dato es Complejo. El problema es que sólo podemos especializar en un método el tipo de la pila (T) y no los tipos genéricos (H, H1, H2) que estos métodos van a recibir.

Por tanto, la solución es indicar en la Pila genérica tantos métodos como posibilidades haya:

Para verCimaDe() son 2: (H) y (Complejo)

```
template <class T>
class Pila {
...
    template <class H1, class H2> static void mostrarPilas(const Pila<H1> &p1, const Pila<H2> &p2);
    template <class H> static void verCimaDe(const Pila<H> &p);
    static void verCimaDe(const Pila<Complejo> &p);
...
};

template <class T>
template <class H1, class H2> void Pila<T>::mostrarPilas (const Pila<H1> &p1, const Pila<H2> &p2) {
    p1.ver(); cout<< ", ";
    p2.ver();
}

template <class T> template <class H>
void Pila<T>::verCimaDe(const Pila<H> &p) { //cuando al método estatico se le pasa
    cout << p.cima(); //una pila de cualquier tipo ejecuta esto
}

template <class T>
void Pila<T>::verCimaDe(const Pila<Complejo> &p) { //cuando al método estatico se le pasa
    p.cima().ver(); //una pila de Complejo ejecuta esto
}
```

De esta forma el código completo queda así:

```
#include <iostream> // std::cout, std::fixed
#include <cstdlib> //system("pause");
#include <cstring> //strlen, strcmp, strcpy

using namespace std;

class Complejo {
    int real;
    int imaginario;
public:
    Complejo(int real=0, int imaginario=0);
    virtual ~Complejo() { }
    int getr() const { return real; }
    void set(int r, int i) { real=r; imaginario=i; }
    int geti() const { return imaginario; }
    void ver() const;
};

Complejo::Complejo(int real, int imaginario) {
    this->real=real;
    this->imaginario=imaginario;
}

void Complejo::ver() const {
    if (imaginario >= 0)
        cout << real << "+" << imaginario << "i";
    else
        cout << real << imaginario << "i";
}

template <class T>
class Pila {
    T *t; //tabla dinamica de elementos
    int n; //numero de elementos del array
    int nmax; //capacidad del array
protected:
    int ampliar(); //metodo para duplicar el tamaño del array dinamico apuntado por t
public:
    Pila(int tam=4); //tamaño opcional, si no se indica ningun valor se toma tamaño 4
    virtual ~Pila(); //destructor
    Pila(const Pila<T>& p); //constructor de copia
    Pila<T>& operator=(const Pila<T>& p); //operador de asignacion
    int longitud() const { return n; }
    int capacidad() const { return nmax; }
    void apilar(T valor);
    void desapilar();
    T cima() const;
    bool esvacia() const { return n==0; }
    virtual void ver() const;

    template <class H1, class H2> static void mostrarPilas(const Pila<H1> &p1, const Pila<H2> &p2);

    template <class H> static void verCimaDe(const Pila<H> &p);
    static void verCimaDe(const Pila<Complejo> &p);

    template <class H> friend ostream& operator<<(ostream& s, const Pila<H> &p);
};

template <class H> ostream& operator<<(ostream& s, const Pila<H> &p);

template class Pila<int>;
template class Pila<float>;
template class Pila<Complejo>;

template <class T>
Pila<T>::Pila(int tam) {
    t=new T[tam]; nmax=tam; n=0;
}

template <class T>
Pila<T>::~~Pila() { delete [] t; }
```



```

template <class T>
Pila<T>::Pila(const Pila<T>& p) {
    t=new T[p.nmax];
    nmax=p.nmax;
    n=p.n;
    for (int i=0; i<n; i++) {
        t[i]=p.t[i];
    }
}

template <class T>
Pila<T>& Pila<T>::operator=(const Pila<T>& p) {
    if (this != &p) {
        delete [] t;
        t=new T[p.nmax];
        nmax=p.nmax;    n=p.n;
        for (int i=0; i<n; i++)
            t[i]=p.t[i];
    }
    return *this;
}

template <class T>
void Pila<T>::apilar(T valor) {
    if (n==nmax)
        ampliar();
    t[n]=valor;
    n++;
}

template <class T>
void Pila<T>::desapilar() {
    if (n==0) {
        cout << "Error: No hay elementos" << endl;
        system("pause");    exit(1);
    }
    n--;
}

template <class T>
T Pila<T>::cima() const {
    if (n==0) {
        cout << "Error: No hay elementos" << endl;
        system("pause");    exit(1);
    }
    return t[n-1];
}

template <class T>
int Pila<T>::ampliar() {
    T *aux=new T[nmax*2];
    for(int i=0; i<n; i++)
        aux[i]=t[i];
    delete [] t;
    t=aux;
    nmax*=2;
    return nmax;
}

template <class T>
void Pila<T>::ver() const { //método genérico ver() (sirve para int y float)
    cout << "[";
    if (n>0) {
        for(int i=0; i<n-1; i++)
            cout << t[i] << " ";
        cout << t[n-1];
    }
    cout << "]";
}

```

```
template <> //especializacion del metodo ver() para el tipo Complejo
```

```
void Pila<Complejo>::ver() const {  
    cout << "[";  
    if (n>0) {  
        for(int i=0; i<n-1; i++) {  
            cout << t[i].getr();  
            if (t[i].geti()>=0)  
                cout << "+";  
            cout << t[i].geti() << "i ";  
        }  
        cout << t[n-1].getr();  
        if (t[n-1].geti()>=0)  
            cout << "+";  
        cout << t[n-1].geti() << "i";  
    }  
    cout << "];"  
}
```

```
template <class T>
```

```
template <class H1, class H2> void Pila<T>::mostrarPilas (const Pila<H1> &p1, const Pila<H2> &p2) {  
    p1.ver(); cout<< ", ";  
    p2.ver();  
}
```

```
template <class T>
```

```
template <class H>
```

```
void Pila<T>::verCimaDe (const Pila<H> &p) { //cuando al método estatico se le pasa  
    cout << p.cima(); //una pila de cualquier tipo ejecuta esto  
}
```

```
template <class T>
```

```
void Pila<T>::verCimaDe (const Pila<Complejo> &p) { //cuando al método estatico se le pasa  
    p.cima().ver(); //una pila de Complejo ejecuta esto  
}
```

```
template <class H> //función genérica operator << (sirve para int y float)
```

```
ostream& operator<<(ostream& s, const Pila<H> &p) {  
    if (p.n==0)  
        s << "(vacío)";  
    for(int i=0; i<p.n; i++) {  
        s << p.t[i] << " ";  
    }  
    return s;  
}
```

```
template <> //especializacion de la funcion operator<< para el tipo Complejo
```

```
ostream& operator<<(ostream& s, const Pila<Complejo> &p) {  
    if (p.n==0)  
        s << "(vacío)";  
    for(int i=0; i<p.n; i++) {  
        s << p.t[i].getr();  
        if (p.t[i].geti()>=0)  
            s << "+";  
        s << p.t[i].geti() << "i ";  
    }  
    return s;  
}
```

```

template <class T>
class PilaDescriptiva : public Pila<T> {
    char *descripcion;
public:
    PilaDescriptiva(int tam, char *texto="Pila");
    PilaDescriptiva(char *texto="Pila");
    virtual ~PilaDescriptiva();
    PilaDescriptiva(const PilaDescriptiva<T>& p);
    PilaDescriptiva& operator=(const PilaDescriptiva<T>& p);

    virtual void ver() const;
    template <class H> friend ostream& operator<<(ostream& s, const PilaDescriptiva<H> &p);
};

template <class H> ostream& operator<<(ostream& s, const PilaDescriptiva<H> &p);

template class PilaDescriptiva<int>;
template class PilaDescriptiva<float>;
template class PilaDescriptiva<Complejo>;

template <class T>
PilaDescriptiva<T>::PilaDescriptiva(int tam, char *texto): Pila<T>(tam) {
    descripcion=new char[strlen(texto)+1];
    strcpy(descripcion, texto);
}

template <class T>
PilaDescriptiva<T>::PilaDescriptiva(char *texto): Pila<T>() {
    descripcion=new char[strlen(texto)+1];
    strcpy(descripcion, texto);
}

template <class T>
PilaDescriptiva<T>::~~PilaDescriptiva() {
    delete [] descripcion;
}

template <class T>
PilaDescriptiva<T>::PilaDescriptiva(const PilaDescriptiva<T>& p): Pila<T>(p) {
    descripcion=new char[strlen(p.descripcion)+1];
    strcpy(descripcion, p.descripcion);
}

template <class T>
PilaDescriptiva<T>& PilaDescriptiva<T>::operator=(const PilaDescriptiva<T>& p) {
    if (this != &p) {
        Pila<T>::operator=(p);
        delete [] descripcion;
        descripcion=new char[strlen(p.descripcion)+1];
        strcpy(descripcion, p.descripcion);
    }
    return *this;
}

template <class T>
void PilaDescriptiva<T>::ver() const {
    cout << descripcion << ": ";
    Pila<T>::ver();
}

template <class H>
ostream& operator<<(ostream& s, const PilaDescriptiva<H> &p) {
    s << p.descripcion << ": " << (Pila<H> &) p;
    return s;
}

int main() {

```

```

Pila<int> p; //crea una pila dinámica de enteros con una capacidad inicial de 4 elementos
//se puede indicar un tamaño diferente Ej: Pila p(8); crea una pila de tamaño 8
Pila<int> q(6); //crea una pila dinámica de enteros de capacidad inicial para 6 elementos
cout << "La pila tiene " << p.longitud() << " elementos" << endl; //indica cuantos elementos hay
cout << "y una capacidad de " << p.capacidad() << " elementos\n"; //indica cuantos caben
cout << "p: ";
p.ver(); //muestra los elementos de la pila entre corchetes
cout << endl;
cout << "p: " << p << endl; //muestra los elementos de la pila
for(int i=1;i<6;i++) {
    p.apilar(i); //mete un elemento en la pila
}
cout << "p: "; p.ver(); cout << endl;
cout << "p: " << p << endl;
cout << p.cima() << endl; //devuelve el elemento de la cima
//de la pila (si no hay ERROR)
Pila<int> r(p); //crea una pila idéntica a la pila p

Pila<Complejo> d; d.apilar(2); d.apilar(5);
cout << "Pilas r y d: "; Pila<float>::mostrarPilas(r,p); cout << endl;
cout << "Pilas r y d: "; Pila<int>::mostrarPilas(r,d); cout << endl;
cout << "Cima de r: "; Pila<char>::verCimaDe(r); cout << endl;
cout << "Cima de d: "; Pila<int>::verCimaDe(d); cout << endl;
//en los métodos estáticos da igual el tipo que pongamos en Pila<tipo>::método_estatico()
p.desapilar(); //elimina el ultimo elemento metido en la pila
p.desapilar();

q=p; //hace que la pila q contenga lo mismo que la pila p
q.apilar(9);
r.apilar(8);

cout << "p: "; p.ver(); cout << endl;
cout << "p: " << p << endl;
cout << p.cima() << endl;
while (!p.esvacía()) { //indica si la pila esta vacia o no
    p.desapilar();
}
cout << "p: "; p.ver(); cout << endl;
cout << "q: "; q.ver(); cout << endl;
cout << "r: "; r.ver(); cout << endl;

PilaDescriptiva<Complejo> pd("Puntos"); //crea pila dinámica de Complejo de capacidad 4
//el texto describe el contenido de la pila
PilaDescriptiva<Complejo> qd(7); //crea pila dinámica de Complejo de capacidad inicial 7 elementos
//el texto explicativo es opcional (si no se indica será "Pila")
cout << "La pila pd y qd tienen " << pd.longitud() << " y " << qd.longitud() << " items" << endl;
cout << "y una capacidad de " << pd.capacidad() << " y " << qd.capacidad() << " elementos\n";
cout << "pd es "; pd.ver(); cout << endl;
cout << "qd es " << qd << endl;
if (!pd.esvacía())
    pd.cima().ver(); //cout << pd.cima(); (no sirve porque Complejo no implementa el operator <<)
for(int i=1;i<6;i++) {
    pd.apilar(i);
}
PilaDescriptiva<Complejo> rd(pd); //crea una pila idéntica a la pila pd

pd.desapilar();
qd=pd;
qd.apilar(8);
cout << pd << endl; //muestra la descripción de la pila: y muestra su contenido
cout << "pd es "; pd.ver(); cout << endl; //muestra descripción y el contenido entre corchetes
cout << "qd es "; qd.ver(); cout << endl;
cout << "rd es "; rd.ver(); cout << endl;

PilaDescriptiva::mostrarPilas(qd, rd); cout << endl;
pd.cima().ver(); cout << endl; // cout << pd.cima() << endl; (no sirve Complejo no operator <<)
return 0;
}

```

- c) Indica que método o función habría que añadir a la clase **Complejo** para que en la clase genérica **Pila** y **PilaDescriptiva** creada en el apartado anterior **no fuera necesario especializar nada**.

En la clase **PilaDescriptiva<T>** no tuvimos que especializar ningún método (no era necesario). En la clase **Pila<T>**, debido a que la clase **Complejo** no tenía implementado el operador **<<**, tuvimos que especializar los métodos que lo usaban: **ver()** y **operator<<**, además de indicar dos versiones (no son especializaciones) para **verCimaDe()**.

Por tanto para no tener que especializar nada en **Pila<T>** **debemos implementar la función operator << en la clase Complejo** (la declaramos amiga de la clase o no, según nos convenga).

```
using namespace std;

class Complejo {
    int real;
    int imaginario;
public:
    Complejo(int real=0, int imaginario=0);
    virtual ~Complejo() { }
    int getr() const { return real; }
    void set(int r, int i) { real=r; imaginario=i; }
    int geti() const { return imaginario; }
    void ver() const;
};

ostream& operator<<(ostream& s, const Complejo& c); //operator << para Complejo

Complejo::Complejo(int real, int imaginario) {
    this->real=real;
    this->imaginario=imaginario;
}

void Complejo::ver() const {
    if (imaginario >= 0)
        cout << real << "+" << imaginario << "i";
    else
        cout << real << imaginario << "i";
}

ostream& operator<<(ostream& s, const Complejo& c) { //implementación operator << para Complejo
    if (c.geti() >= 0)
        s << c.getr() << "+" << c.geti() << "i";
    else
        s << c.getr() << c.geti() << "i";
    return s;
}
```

De esta forma la clase **Pila** genérica queda así (y en el .cpp no se especializa ningún método):

```
template <class T>
class Pila {
...
    template <class H1, class H2> static void mostrarPilas(const Pila<H1> &p1, const Pila<H2> &p2);
    template <class H> static void verCimaDe(const Pila<H> &p);
...
};

template <class T>
template <class H1, class H2> void Pila<T>::mostrarPilas (const Pila<H1> &p1, const Pila<H2> &p2) {
    p1.ver(); cout<< ", ";
    p2.ver();
}

template <class T> template <class H>
void Pila<T>::verCimaDe(const Pila<H> &p) {
    cout << p.cima(); //←ya no da error porque << funciona con int, float y Complejo
}
```